



K-Nearest Neighbors (KNN)

Aprendizaje basado en instancias, métricas y regiones locales

Círculo de Ciencia de Datos y Econometría

19 de septiembre de 2026

Mapa de la presentación

1. Fundamentos e Intuición de KNN
2. Formulación Matemática de KNN
3. Métricas de Distancia y Geometría
4. Escalamiento e Hiperparámetros
5. Maldición de la Dimensionalidad y Complejidad
6. Implementación Reproducible en Python
7. Cierre y Fuentes

Pregunta guía

¿Cómo puede un algoritmo clasificar o predecir sin asumir una función paramétrica global, aprendiendo directamente de la geometría local de los datos?

Fuentes principales

James et al. (ISLR, cap. 2 y 4); Hastie et al. (ESL, cap. 13); Géron (2022, cap. 3).

Aplicación práctica

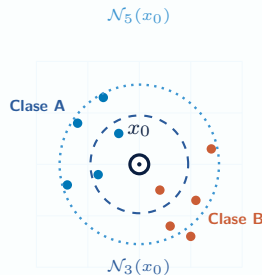
Python + scikit-learn con Pipeline reproducible.

¿Qué es K-Nearest Neighbors (KNN)?

K-Nearest Neighbors (KNN) es un algoritmo supervisado **no paramétrico** y **basado en instancias**.

- ▶ **Sin parámetros globales:** no calcula un vector β .
- ▶ **Lazy learning:** memoriza el conjunto de entrenamiento.
- ▶ **Fronteras locales:** aproxima relaciones no lineales.

Regla de decisión: la predicción para x_0 se obtiene consultando el vecindario local $\mathcal{N}_k(x_0)$.



Fuente: James et al. (2013), cap. 2; Hastie et al. (2009), cap. 13.

Paradigma: Modelos Paramétricos vs. Basados en Instancias

Enfoque Paramétrico

Ejemplos: Regresión lineal, Logit, LDA.

- ▶ Asume forma funcional global explícita:

$$f(X) = \beta_0 + \sum_{j=1}^p \beta_j X_j$$

- ▶ **Entrenamiento:** intensivo (optimización).
- ▶ **Inferencia:** instantánea con $\hat{\beta}$.
- ▶ **Riesgo:** sesgo por mala especificación.

Enfoque No Paramétrico (KNN)

Ejemplos: KNN, Splines, Kernel regression.

- ▶ Sin estructura funcional previa:

$$\hat{f}(x_0) = \text{Agreg}(\{y_i : x_i \in \mathcal{N}_k(x_0)\})$$

- ▶ **Entrenamiento:** pasivo (memorización).
- ▶ **Inferencia:** costo $O(N \cdot p)$ por consulta.
- ▶ **Riesgo:** alta varianza en alta dimensión.

Fuente: Géron (2022), cap. 3; James et al. (2013), cap. 2.

KNN en Clasificación: Voto Mayoritario y Probabilidad

Definición de Vecindad Local

Sea $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ y x_0 un punto de consulta. $\mathcal{N}_k(x_0)$ contiene los k puntos más cercanos a x_0 según $d(\cdot, \cdot)$.

Probabilidad a Posteriori

Frecuencia empírica en el vecindario para la clase $c \in \mathcal{C}$:

$$\hat{P}(Y = c \mid X = x_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_0)} \mathbb{I}(y_i = c)$$

Regla de Voto Mayoritario

Asigna la clase con mayor soporte en $\mathcal{N}_k(x_0)$:

$$\hat{y}(x_0) = \arg \max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(x_0)} \mathbb{I}(y_i = c)$$

Teorema de Cover y Hart (1967):

Cuando $N \rightarrow \infty$, la tasa de error del 1-NN satisface:

$$R^* \leq R_{1\text{-NN}} \leq 2R^*(1 - R^*)$$

donde R^* es el error del clasificador óptimo de Bayes.

Fuente: Cover & Hart (1967); James et al. (2013), cap. 2.

KNN en Regresión y Ponderación por Distancia

KNN Regresión Estándar

Para $y_i \in \mathbb{R}$, media local de vecinos:

$$\hat{f}(x_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_0)} y_i$$

- ▶ Función escalonada por tramos.
- ▶ Ponderación uniforme ($1/k$) en $\mathcal{N}_k(x_0)$.

Ponderación por Distancia

Mayor influencia a observaciones próximas:

$$w_i = \frac{1}{d(x_0, x_i)^p}, \quad \hat{f}(x_0) = \frac{\sum_{i \in \mathcal{N}_k} w_i y_i}{\sum_{i \in \mathcal{N}_k} w_i}$$

- ▶ Genera fronteras y ajustes suaves.
- ▶ Atenúa el impacto de puntos lejanos.

Interpretación: KNN ponderado equivale a un **estimador de Nadaraya-Watson** con ancho de banda adaptativo a la distancia del k -ésimo vecino.

Fuente: Hastie, Tibshirani & Friedman (2009), cap. 6 y 13.

Métricas de Distancia en $\mathbb{R}^p$

Métrica	Fórmula	Propiedades y Uso Práctico
Euclidiana (L_2)	$\sqrt{\sum_{j=1}^p (x_j - z_j)^2}$	Distancia geodésica estándar en $\mathbb{R}^p$. Asume isotropía.
Manhattan (L_1)	$\sum_{j=1}^p x_j - z_j $	Distancia taxicab. Más robusta ante valores atípicos que L_2 .
Minkowski (L_q)	$\left(\sum_{j=1}^p x_j - z_j ^q\right)^{\frac{1}{q}}$	Generaliza L_1 ($q = 1$) y L_2 ($q = 2$).
Chebyshev (L_∞)	$\max_j x_j - z_j $	Dominada por la dimensión de mayor discrepancia.
Hamming	$\sum_{j=1}^p \mathbb{I}(x_j \neq z_j)$	Para variables binarias o categóricas discretas.

Conclusión: La métrica define formalmente el concepto de vecindad. Modificar la distancia altera el orden relativo de los vecinos y puede cambiar la predicción manteniendo k fijo.

Fuente: Géron (2022), cap. 3; Hastie et al. (2009), cap. 13.

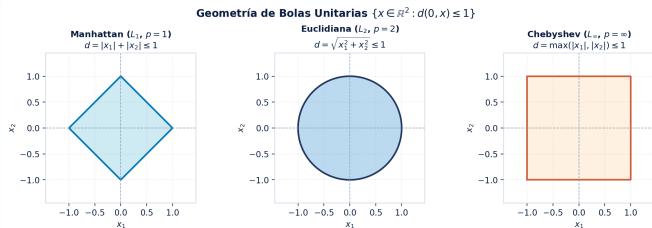
Geometría de Bolas Unitarias en 2D

Lectura Geométrica

Bolas unitarias $\{x : d(0, x) \leq 1\}$:

- ▶ **Manhattan (L_1):** rombo orientado a 45° .
- ▶ **Euclidiana (L_2):** circunferencia perfecta.
- ▶ **Chebyshev (L_∞):** cuadrado ortogonal.

Efecto en la frontera: al pasar de L_2 a L_1 o L_∞ , las fronteras de decisión se vuelven poligonales.

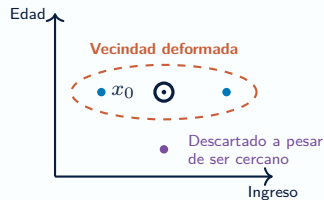


Fuente: Figura reproducible generada con Python scikit-learn y matplotlib (CCDE).

Por Qué el Escalamiento es Crítico en KNN

Como KNN mide distancias absolutas en $\mathbb{R}^p$, variables con mayor escala numérica dominan arbitrariamente la cercanía.

- ▶ **Ejemplo:** Ingreso mensual (\$1 000 a \$20 000) vs. Edad (18 a 80).
- ▶ Diferencia de \$100 eclipsa totalmente 40 años de edad.
- ▶ **Efecto:** la distancia ignora las variables de menor escala.



Solución con StandardScaler

Estandarizar ($z_j = \frac{x_j - \mu_j}{\sigma_j}$) en un Pipeline para evitar fuga de datos (*leakage*).

Fuente: Géron (2022), cap. 3; James et al. (2013), cap. 4.

El Papel de k : Trade-Off Sesgo–Varianza

k Pequeño: Flexibilidad

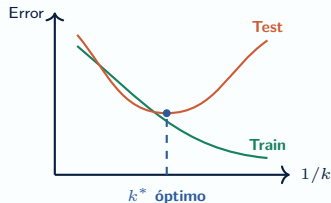
Bajo sesgo / Alta varianza ($k = 1$)

- ▶ Reproduce fielmente el training set.
- ▶ Frontera fragmentada, sensible a anomalías.

k Grande: Regularización

Alto sesgo / Baja varianza ($k \rightarrow N$)

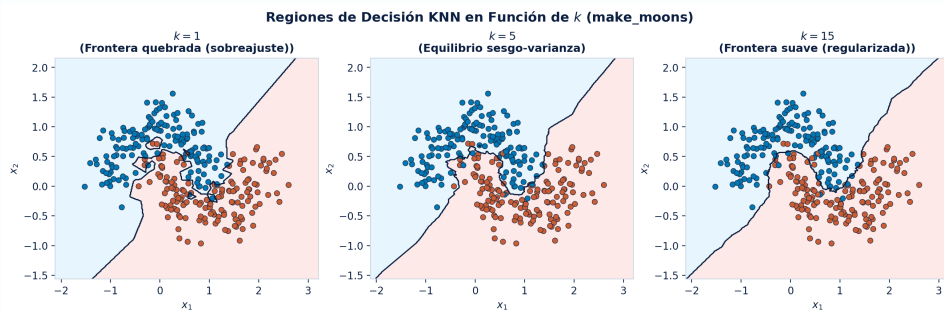
- ▶ Fronteras excesivamente suaves.
- ▶ Predice la clase mayoritaria global.



Regla: el hiperparámetro k se optimiza mediante **validación cruzada (CV)**.

Fuente: Hastie, Tibshirani & Friedman (2009), cap. 2 y 13.

Evolución de las Regiones de Decisión según k



$k = 1$ (Sobreajuste):

Frontera hiperflexible; islas espurias alrededor de puntos ruidosos.

$k = 5$ (Balance Óptimo):

Captura la curvatura no lineal de las lunas respetando la estructura.

$k = 15$ (Regularizado):

Frontera suave y estable; reduce la varianza minimizando el sobreajuste.

Fuente: Figura generada con scikit-learn (make_moons) y KNeighborsClassifier.

La Maldición de la Dimensionalidad (*Curse of Dimensionality*)

El Hipervolumen Vacío

En $[0, 1]^p$, capturar una fracción $r \in (0, 1)$ del volumen total exige una arista local:

$$e_p(r) = r^{\frac{1}{p}}$$

- ▶ $p = 1$: $e_1(0,10) = 0,10$ (10 % del rango).
- ▶ $p = 10$: $e_{10}(0,10) \approx 0,80$ (80 % del rango).
- ▶ $p = 100$: $e_{100}(0,10) \approx 0,98$ (98 % del rango).

Consecuencia Teórica para KNN:

En alta dimensión, una vecindad de k puntos **deja de ser local**: abarca casi todo el espacio muestral.

Equidistancia Asintótica:

$$\lim_{p \rightarrow \infty} \frac{d_{\text{máx}} - d_{\text{mín}}}{d_{\text{mín}}} = 0$$

Todos los puntos quedan aproximadamente a la misma distancia, diluyendo el concepto de cercanía.

Fuente: Hastie, Tibshirani & Friedman (2009), cap. 2; Donoho (2000).

Algoritmos de Búsqueda

- ▶ **Fuerza Bruta (brute)**: $O(N \cdot p)$ por consulta; evalúa todas las distancias.
- ▶ **KD-Tree (kd_tree)**: Partición ortogonal; eficiente con $p \leq 20$ ($O(p \log N)$).
- ▶ **Ball-Tree (ball_tree)**: Hiperesferas anidadas; óptimo para métricas no euclidianas.

Balance Metodológico

Fortalezas:

- ▶ Intuitivo y no paramétrico.
- ▶ Excelente *baseline* no lineal.
- ▶ Extensión natural a multiclase.

Limitaciones:

- ▶ Latencia en inferencia ($O(N \cdot p)$).
- ▶ Sensible a variables irrelevantes.
- ▶ Almacenamiento continuo de datos.

Fuente: Bentley (1975); Omohundro (1989); scikit-learn documentation.

Pipeline con Scikit-Learn: Prevención de Data Leakage

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

# Pipeline estricto: escalamiento + estimador
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
```

Fuente: Script reproducible `codigo_demo_knn.py` (CCDE).

Arquitectura Pipeline:

- ▶ Aplica μ y σ aprendidos **solo en el pliegue de entrenamiento**.
- ▶ Evita fuga de información (*data leakage*) al test/validación.
- ▶ Facilita la reproducibilidad integral del flujo.

Dataset: Breast Cancer Wisconsin (569 obs., 30 features numéricas).

Búsqueda de Hiperparámetros con GridSearchCV

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 11, 15],
    "knn__weights": ["uniform", "distance"],
    "knn__metric": ["minkowski"],
    "knn__p": [1, 2] # 1: Manhattan, 2: Euclidiana
}

grid = GridSearchCV(
    pipe,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
grid.fit(X_tr, y_tr)
best_knn = grid.best_estimator_
```

Fuente: Pedregosa et al. (2011); scikit-learn documentation.

Espacio de Búsqueda

- ▶ **Vecindades (k):** de 3 a 15 impares para evitar empates.
- ▶ **Ponderación:** uniforme vs inversa a la distancia.
- ▶ **Métricas:** norma L_1 vs L_2 .
- ▶ **Criterio:** ROC-AUC con 5 pliegues estratificados.

Resultados Empíricos y Matriz de Confusión

Mejor Configuración Encontrada

Parámetro / Métrica	Valor
k óptimo (n_neighbors)	15
Ponderación (weights)	distance
Métrica	Euclidiana (L_2)
ROC-AUC CV (5-fold)	0.9875
Accuracy en Test	96.50 %
ROC-AUC en Test	0.9948

Matriz de Confusión en Test

	Pred. Mal.	Pred. Ben.
Real Maligno	48	5
Real Benigno	0	90

- ▶ Sensibilidad (Recall benigno): 100 %.
- ▶ Especificidad (clase maligna): 90.57 %.

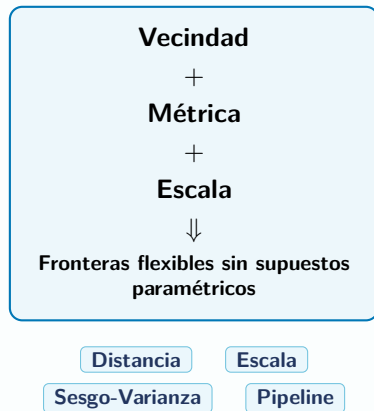
Conclusión: La ponderación por distancia combinada con $k = 15$ maximizó la generalización en test.

Fuente: Resultados reproducibles obtenidos con `codigo_demo_knn.py` (semilla 42).

Ideas Clave que Deben Quedar

1. **Aprendizaje no paramétrico:** KNN no asume una función global $\hat{f}(X)$; aproxima la densidad condicional evaluando vecindades locales $\mathcal{N}_k(x_0)$.
2. **La métrica define la vecindad:** cambiar la distancia (Euclidiana, Manhattan, Chebyshev) altera la geometría de las regiones de decisión.
3. **Escalamiento obligatorio:** diferencias de escala dominan la distancia; usar siempre un Pipeline con StandardScaler.
4. **Hiperparámetro k :** controla el trade-off sesgo-varianza ($k = 1$ sobreajusta; k grande suaviza en exceso).
5. **Maldición de la dimensionalidad:** en alta dimensión los puntos se vuelven equidistantes y el soporte local deja de ser representativo.
6. **Lazy learner:** entrenamiento trivial $O(1)$, pero costo computacional desplazado a la inferencia $O(N \cdot p)$.

Fuente: James et al. (2013); Hastie et al. (2009); Géron (2022).



Bibliografía Base Recomendada

James, G.; Witten, D.; Hastie, T.; Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.

Capítulos 2 y 4: fundamentos de aprendizaje supervisado, clasificador de Bayes, formulación de KNN y trade-off sesgo-varianza.

Hastie, T.; Tibshirani, R.; Friedman, J. (2009). *The Elements of Statistical Learning*, 2.^a ed., Springer.

Capítulos 2 y 13: teoría de estimadores locales por densidad de kernel, prototipos, tasas de error y maldición de la dimensionalidad.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3.^a ed., O'Reilly.

Capítulo 3: implementación práctica de `KNeighborsClassifier`, escalamiento y validación con `Pipeline` y `GridSearchCV`.